

GIẢI THÍCH TOÀN BỘ BACKEND: CHỨC NĂNG THỐNG KÊ PHẢN HỒI

Giải thích từng file, từng hàm, hàm nào gọi hàm nào

SƠ ĐỒ GỌI HÀM — ĐỌC CÁI NÀY TRƯỚC



FILE 1: `application.properties` — CẤU HÌNH DỊCH VỤ

```
spring.application.name=feedback-service
server.port=8083 # Chạy ở cổng 8083

# Kết nối database MySQL riêng của feedback-service
spring.datasource.url=jdbc:mysql://localhost:3308/feedback_db?createDatabaseIfNotExist=true...
# localhost:3308 → cổng 3308 (Docker, khác với product-service dùng 3307)
# feedback_db → database riêng, hoàn toàn tách biệt

spring.jpa.hibernate.ddl-auto=update # Tự động tạo/cập nhật bảng theo Entity

# URL để gọi sang các service khác
product-service.url=${PRODUCT_SERVICE_URL:http://localhost:8081}
# ${PRODUCT_SERVICE_URL:http://localhost:8081} nghĩa là:
# Nếu có biến môi trường PRODUCT_SERVICE_URL → dùng nó (khi deploy Docker/server)
# Nếu không có → dùng giá trị mặc định http://localhost:8081 (khi chạy local)
user-service.url=${USER_SERVICE_URL:http://localhost:8082}
```

FILE 2: `RestTemplateConfig.java` — CẤU HÌNH CÔNG CỤ GỌI HTTP

```

@Configuration // ← File cấu hình, Spring đọc khi khởi động
public class RestTemplateConfig {

    @Bean // ← Tạo object này và đưa vào "kho" của Spring (Spring Container)
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
    // RestTemplate là công cụ của Spring để gọi HTTP request ra bên ngoài
    // Giống fetch() trong JavaScript
    //
    // Tại sao phải khai báo @Bean ở đây?
    // Vì UserServiceClient và ProductServiceClient cần dùng RestTemplate
    // Thay vì tự new RestTemplate() trong mỗi class → khai báo 1 lần ở đây
    // Spring tự inject vào bất kỳ class nào cần
}

```

FILE 3: Feedback.java — ENTITY PHẢN HỒI

```

@Entity
@Table(name = "feedbacks")
public class Feedback {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long productId;
    // Chỉ lưu ID số nguyên, KHÔNG có khóa ngoại thực sự sang bảng products
    // Vì products nằm trong database KHÁC (product_db), không thể tạo FK
    // Đây gọi là "soft reference" (tham chiếu mềm) – chỉ biết ID, không biết object

    @Column(nullable = false)
    private Long customerId; // Tương tự – ID của khách hàng trong user_db

    @Column(length = 1000)
    private String comment; // Nhận xét tổng thể, tối đa 1000 ký tự

    @Column(name = "rating", nullable = false)
    private Integer overallRating; // Điểm tổng: 1, 2, 3, 4, hoặc 5

    @Column(nullable = false)
    private LocalDateTime createdAt = LocalDateTime.now();
    // LocalDateTime.now() chạy khi new Feedback() → tự lấy thời điểm hiện tại
    // Ví dụ: 2025-03-15T10:30:00

    @OneToMany(
        mappedBy = "feedback",
        cascade = CascadeType.ALL, // lưu/xóa Feedback → tự lưu/xóa AttributeRating
        orphanRemoval = true,
        fetch = FetchType.EAGER // load Feedback → load luôn danh sách AttributeRating
    )
    private List<AttributeRating> attributeRatings = new ArrayList<>();
    // 1 feedback có thể có nhiều đánh giá thuộc tính
    // Ví dụ: Feedback về iPhone 15 có đánh giá Màu sắc 5*, RAM 4*, Pin 3*

    @Transient // ← KHÔNG lưu vào DB
    private String customerName;
    // Field này tồn tại trong object Java nhưng KHÔNG có cột tương ứng trong DB
}

```

```
// Được điền bởi FeedbackService.enrich() sau khi gọi sang user-service
// Mỗi lần query → mỗi lần enrich() lại – không bao giờ stale
}
```

FILE 4: `AttributeRating.java` — ENTITY ĐÁNH GIÁ THUỘC TÍNH

Mỗi feedback có thể đánh giá nhiều thuộc tính của sản phẩm. File này lưu từng đánh giá đó.

```
@Entity
@Table(name = "attribute_ratings")
public class AttributeRating {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore // ← Ẩn khỏi JSON để tránh vòng lặp vô tận
    @ManyToOne(optional = false)
    @JoinColumn(name = "feedback_id") // cột "feedback_id" là khóa ngoại → bảng feedbacks
    private Feedback feedback;
    // Nhiều AttributeRating → 1 Feedback
    // @JsonIgnore vì: serialize AttributeRating → serialize Feedback
    // → serialize List<AttributeRating> của Feedback → vòng lặp vô tận

    @Column(nullable = false)
    private Long attributeId;
    // ID của thuộc tính trong product_db (product-service)
    // Không có FK thực vì khác database

    @Column(nullable = false)
    private Integer rating; // Điểm của thuộc tính này: 1-5

    @Column(length = 500)
    private String comment; // Nhận xét riêng cho thuộc tính này (tùy chọn)

    @Transient
    private String attributeName;
    // Không lưu DB – được điền bởi enrich() sau khi gọi sang product-service
    // Ví dụ: attributeId=3, attributeName="Màu sắc"

    public AttributeRating() {} // Constructor rỗng – bắt buộc cho JPA

    // Constructor tiện lợi khi tạo mới
    public AttributeRating(Feedback feedback, Long attributeId, Integer rating, String comment) {
        this.feedback = feedback;
        this.attributeId = attributeId;
        this.rating = rating;
        this.comment = comment;
    }
}
```

Mối quan hệ giữa các bảng trong `feedback_db`:

Bảng `feedbacks`:

id	productId	customerId	comment	rating	createdAt
1	5	12	Tốt lắm	4	2025-03-15 10:30:00

	2		5		15		Bình thường		3		2025-03-16 09:00:00	
+	-----	+	-----	+	-----	+	-----	+	-----	+	-----	+
Bảng attribute_ratings:												
+	-----	+	-----	+	-----	+	-----	+	-----	+	-----	+
	id		feedback_id		attributeId		rating		comment			
+	-----	+	-----	+	-----	+	-----	+	-----	+		
	1		1		3		5		Màu đẹp		← feedback 1 đánh giá thuộc tính 3 (Màu sắc)	
	2		1		4		4		RAM ổn		← feedback 1 đánh giá thuộc tính 4 (RAM)	
	3		2		3		3		Màu tạm		← feedback 2 đánh giá thuộc tính 3 (Màu sắc)	
+	-----	+	-----	+	-----	+	-----	+	-----	+		

FILE 5: FeedbackRepository.java — KHO TRUY VẤN

```
public interface FeedbackRepository extends JpaRepository<Feedback, Long> {
// JpaRepository cho miễn phí: save(), findById(), deleteById(), existsById(), count()...

// — HÀM 1: Lấy feedback trong khoảng ngày (tất cả sản phẩm) —————
@Query("SELECT f FROM Feedback f " +
        "WHERE f.createdAt BETWEEN :from AND :to " +
        "ORDER BY f.createdAt DESC")
List<Feedback> findByDateRange(
        @Param("from") LocalDateTime from,
        @Param("to")    LocalDateTime to);
// BETWEEN :from AND :to → createdAt >= from AND createdAt <= to
// SQL thực tế:
//   SELECT * FROM feedbacks
//   WHERE created_at BETWEEN '2025-01-01 00:00:00' AND '2025-12-31 23:59:59'
//   ORDER BY created_at DESC
//
// Vì FetchType.EAGER trên attributeRatings → Spring tự JOIN thêm:
//   LEFT JOIN attribute_ratings ar ON ar.feedback_id = f.id

// — HÀM 2: Lấy feedback của 1 sản phẩm trong khoảng ngày —————
@Query("SELECT f FROM Feedback f " +
        "WHERE f.productId = :productId " +
        "AND f.createdAt BETWEEN :from AND :to " +
        "ORDER BY f.createdAt DESC")
List<Feedback> findByProductIdAndDateRange(
        @Param("productId") Long productId,
        @Param("from")      LocalDateTime from,
        @Param("to")        LocalDateTime to);
// Thêm điều kiện lọc theo productId so với hàm trên
// SQL: SELECT * FROM feedbacks
//       WHERE product_id = 5
//       AND created_at BETWEEN '...' AND '...'
//       ORDER BY created_at DESC
}
```

FILE 6: FeedbackController.java — CONTROLLER (BỒI BÀN)

```
@RestController
@RequestMapping("/api/feedbacks")
public class FeedbackController {
```

```

private final FeedbackService feedbackService;

public FeedbackController(FeedbackService feedbackService) {
    this.feedbackService = feedbackService;
}

// — HÀM 1: Lấy 1 feedback theo ID —————
@GetMapping("/{id}") // ← GET /api/feedbacks/1
public Feedback getById(@PathVariable Long id) {
    log.info("GET /api/feedbacks/{id}", id);
    return feedbackService.getFeedbackById(id);
    // Trả thẳng Feedback (không bọc ResponseEntity) → Spring tự chuyển sang JSON
}

// — HÀM 2: Lấy feedback theo khoảng thời gian —————
@GetMapping("/range") // ← GET /api/feedbacks/range
public List<Feedback> getByDateRange(
    @RequestParam String from, // ?from=2025-01-01
    @RequestParam String to, // &to=2025-12-31
    @RequestParam(required = false) Long productId) { // &productId=5 (không bắt buộc)
    // required = false → nếu không gửi productId trong URL → productId = null
    // Khi productId = null → lấy tất cả sản phẩm
    // Khi productId = 5 → chỉ lấy feedback của sản phẩm id=5

    log.info("GET /api/feedbacks/range from={id} to={id} productId={id}", from, to, productId);
    return feedbackService.getFeedbacksByDateRange(from, to, productId);
    // Controller không làm gì ngoài nhận request và chuyển cho Service
}

// — HÀM 3: Xóa feedback —————
@DeleteMapping("/{id}") // ← DELETE /api/feedbacks/1
public ResponseEntity<Void> delete(@PathVariable Long id) {
    log.info("[ADMIN] DELETE /api/feedbacks/{id}", id);
    feedbackService.deleteFeedback(id);
    return ResponseEntity.noContent().build(); // HTTP 204 No Content
}
}

```

FILE 7: FeedbackService.java — SERVICE (TRÁI TIM)

```

@Service
public class FeedbackService {

    private final FeedbackRepository feedbackRepository;
    private final ProductServiceClient productServiceClient;
    private final UserServiceClient userServiceClient;

    public FeedbackService(FeedbackRepository feedbackRepository,
        ProductServiceClient productServiceClient,
        UserServiceClient userServiceClient) {
        this.feedbackRepository = feedbackRepository;
        this.productServiceClient = productServiceClient;
        this.userServiceClient = userServiceClient;
    }

    // — HÀM 1: Lấy 1 feedback theo ID —————
    @Transactional(readOnly = true)
    public Feedback getFeedbackById(Long id) {
        Feedback fb = feedbackRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Feedback không tồn tại: id=" +

```

```

id));
    enrich(List.of(fb)); // List.of(fb) = tạo list chứa 1 phần tử
    // Gọi enrich() để điền customerName và attributeName
    return fb;
}

// — HÀM 2: Lấy feedback theo khoảng ngày —————
@Transactional(readonly = true)
public List<Feedback> getFeedbacksByDateRange(String fromDate, String toDate, Long productId) {

    // Bước 1: Chuyển chuỗi "2025-01-01" thành LocalDateTime
    LocalDateTime from = LocalDate.parse(fromDate).atStartOfDay();
    // LocalDate.parse("2025-01-01") → LocalDate(2025, 1, 1)
    // .atStartOfDay() → LocalDateTime(2025, 1, 1, 0, 0, 0) = 00:00:00

    LocalDateTime to = LocalDate.parse(toDate).atTime(LocalTime.MAX);
    // .atTime(LocalTime.MAX) → LocalDateTime(2025, 12, 31, 23, 59, 59, 999999999)
    // = 23:59:59 cuối ngày → lấy hết feedback trong ngày cuối

    // Bước 2: Truy vấn DB
    List<Feedback> list = productId != null
        ? feedbackRepository.findByProductIdAndDateRange(productId, from, to)
        : feedbackRepository.findByDateRange(from, to);
    // Toán tử 3 ngôi: nếu có productId → lọc theo sản phẩm, không thì lấy tất cả

    // Bước 3: Bổ sung tên rồi trả về
    return enrich(list);
}

// — HÀM 3: enrich() – BỔ SUNG TÊN TỪ CÁC SERVICE KHÁC —————
private List<Feedback> enrich(List<Feedback> list) {
    // Tạo 2 bộ nhớ đệm (cache) để tránh gọi HTTP trùng lặp
    Map<Long, String> customerCache = new java.util.HashMap<>();
    Map<Long, String> attributeCache = new java.util.HashMap<>();
    // HashMap: cấu trúc dữ liệu "từ điển" – lưu cặp key → value
    // Tìm theo key rất nhanh: O(1)

    for (Feedback fb : list) { // duyệt qua từng feedback

        // == Lấy tên khách hàng ==
        String cname = customerCache.computeIfAbsent(fb.getCustomerId(), cid -> {
            // ↑ key                                ↑ lambda function
            // computeIfAbsent(key, function):
            // Nếu key ĐÃ có trong Map → trả về value đã lưu (không chạy lambda)
            // Nếu key CHƯA có → chạy lambda, lưu kết quả vào Map, trả về kết quả

            try {
                com.example.feedback.dto.CustomerDto c = userServiceClient.getCustomer(cid);
                // Gọi HTTP sang user-service – xem File 8 bên dưới
                return (c != null && c.getFullName() != null) ? c.getFullName() : null;
            } catch (Exception e) {
                return null; // Nếu user-service chết → trả null, không crash
            }
        });
        fb.setCustomerName(cname);
        // Gắn tên vào field @Transient – KHÔNG lưu DB

        // == Lấy tên thuộc tính cho từng AttributeRating ==
        for (com.example.feedback.entity.AttributeRating ar : fb.getAttributeRatings()) {
            String aname = attributeCache.computeIfAbsent(ar.getAttributeId(), aid -> {
                try {
                    com.example.feedback.dto.AttributeDto a =
productServiceClient.getAttribute(aid);
                    // Gọi HTTP sang product-service – xem File 9 bên dưới

```

```

        return (a != null) ? a.getName() : null;
    } catch (Exception e) {
        return null;
    }
    });
    ar.setAttributeName(aname);
    // Gắn tên thuộc tính vào field @Transient
}
}
return list; // Trả về danh sách đã được bổ sung đầy đủ thông tin
}

// — HÀM 4: Xóa feedback —————
@Transactional
public void deleteFeedback(Long id) {
    if (!feedbackRepository.existsById(id)) {
        throw new IllegalArgumentException("Feedback không tồn tại: id=" + id);
    }
    feedbackRepository.deleteById(id);
    // Vì CascadeType.ALL + orphanRemoval:
    // Spring tự DELETE attribute_ratings WHERE feedback_id = 1 trước
    // Sau đó DELETE feedbacks WHERE id = 1

    log.info("[DB] Đã xóa phản hồi id={}", id);
}
}

```

FILE 8: UserServiceClient.java — GỌI SANG USER-SERVICE

```

@Component // Spring quản lý class này, có thể inject vào FeedbackService
public class UserServiceClient {

    @Value("${user-service.url}")
    private String userServiceUrl;
    // @Value đọc từ application.properties:
    // user-service.url=http://localhost:8082
    // Spring inject giá trị này vào field khi khởi động

    private final RestTemplate restTemplate;
    // Được inject từ RestTemplateConfig.restTemplate() (File 2)

    public UserServiceClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    // — HÀM DUY NHẤT: Lấy thông tin khách hàng theo ID —————
    public CustomerDto getCustomer(Long customerId) {

        // Tạo ID ngẫu nhiên để theo dõi request xuyên suốt các service
        String correlationId = UUID.randomUUID().toString();
        // UUID.randomUUID() = tạo chuỗi ngẫu nhiên duy nhất, ví dụ:
        // "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
        // Dùng để ghép log của nhiều service lại khi debug

        String url = userServiceUrl + "/api/customers/" + customerId;
        // Ví dụ: "http://localhost:8082/api/customers/12"

        log.info("[INTER-SERVICE] [CID:{}] feedback-service → user-service | GET {} | customerId=
        {}",
            correlationId, url, customerId);
    }
}

```

```

try {
    // Tạo HTTP headers để gửi kèm
    HttpHeaders headers = new HttpHeaders();
    headers.set("X-Correlation-ID", correlationId);
    // user-service nhận header này và log với cùng ID → dễ trace

    HttpEntity<Void> entity = new HttpEntity<>(headers);
    // HttpEntity<Void>: request không có body (GET không có body)
    // Nhưng có headers

    // Thực sự gọi HTTP GET
    ResponseEntity<CustomerDto> response = restTemplate.exchange(
        url,                // URL gọi đến
        HttpMethod.GET,     // phương thức HTTP
        entity,             // headers (không có body)
        CustomerDto.class   // kiểu dữ liệu parse response body về
    );
    // Spring tự nhận JSON response và parse thành CustomerDto:
    // {"id":12,"fullName":"Nguyễn Văn A","email":"..."} → CustomerDto object

    CustomerDto customer = response.getBody();    // lấy body đã parse

    if (customer != null) {
        log.info("[INTER-SERVICE] [CID:{}] user-service trả về fullName='{}'",
            correlationId, customer.getFullName());
    } else {
        log.warn("[INTER-SERVICE] [CID:{}] user-service trả về null cho customerId={},",
            correlationId, customerId);
    }

    return customer;

} catch (Exception e) {
    // Bắt mọi lỗi: timeout, connection refused, HTTP 4xx/5xx...
    log.error("[INTER-SERVICE] [CID:{}] Lỗi khi gọi user-service | customerId={} | Lỗi:
{}",
        correlationId, customerId, e.getMessage());
    return null;
    // Trả null thay vì để crash → FeedbackService.enrich() sẽ xử lý null
}
}
}

```

FILE 9: ProductServiceClient.java — GỌI SANG PRODUCT-SERVICE

```

@Component
public class ProductServiceClient {

    @Value("${product-service.url}")
    private String productServiceUrl;    // "http://localhost:8081"

    private final RestTemplate restTemplate;

    public ProductServiceClient(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    // — HÀM 1: Lấy thông tin sản phẩm theo ID —————
    public ProductDto getProduct(Long productId) {

```

```

String correlationId = UUID.randomUUID().toString();
String url = productServiceUrl + "/api/products/" + productId;
// Ví dụ: "http://localhost:8081/api/products/5"

try {
    HttpHeaders headers = new HttpHeaders();
    headers.set("X-Correlation-ID", correlationId);
    HttpEntity<Void> entity = new HttpEntity<>(headers);

    ResponseEntity<ProductDto> response = restTemplate.exchange(
        url, HttpMethod.GET, entity, ProductDto.class);
    ProductDto product = response.getBody();

    if (product != null) {
        log.info("[INTER-SERVICE] [CID:{}] product-service trả về id={}, name='{}'",
            correlationId, product.getId(), product.getName());
    }
    return product;
} catch (Exception e) {
    log.error("[INTER-SERVICE] [CID:{}] Lỗi gọi product-service | productId={} | Lỗi: {}",
        correlationId, productId, e.getMessage());
    return null;
}
}

// — HÀM 2: Lấy thông tin thuộc tính theo ID —————
// ĐÂY LÀ HÀM ĐƯỢC DÙNG TRONG enrich()
public AttributeDto getAttribute(Long attributeId) {
    String correlationId = UUID.randomUUID().toString();
    String url = productServiceUrl + "/api/products/attributes/" + attributeId;
    // Ví dụ: "http://localhost:8081/api/products/attributes/3"
    // → Gọi vào ProductController.getAttributeById() bên product-service

    try {
        HttpHeaders headers = new HttpHeaders();
        headers.set("X-Correlation-ID", correlationId);
        HttpEntity<Void> entity = new HttpEntity<>(headers);

        ResponseEntity<AttributeDto> response = restTemplate.exchange(
            url, HttpMethod.GET, entity, AttributeDto.class);
        AttributeDto attribute = response.getBody();
        // Spring parse JSON {"id":3,"name":"Màu sắc"} → AttributeDto object

        if (attribute != null) {
            log.debug("[INTER-SERVICE] [CID:{}] product-service trả về attributeId={},
name='{}'",
                correlationId, attribute.getId(), attribute.getName());
        }
        return attribute;
    } catch (Exception e) {
        log.warn("[INTER-SERVICE] [CID:{}] Lỗi gọi product-service/attributes | attributeId={}
| Lỗi: {}",
            correlationId, attributeId, e.getMessage());
        return null;
    }
}
}

```

LUỒNG DỮ LIỆU ĐẦY ĐỦ VỚI VÍ DỤ CỤ THỂ

Yêu cầu: GET /api/feedbacks/range?from=2025-03-01&to=2025-03-31

Trong DB:

```
feedbacks: [{id:1, productId:5, customerId:12, rating:4, ...}]
attribute_ratings: [{id:1, feedback_id:1, attributeId:3, rating:5}, {id:2, feedback_id:1, attributeId:4, rating:4}]
```

Luồng chạy:

1. FeedbackController.getByDateRange("2025-03-01", "2025-03-31", null) được gọi
2. Gọi FeedbackService.getFeedbacksByDateRange("2025-03-01", "2025-03-31", null)
3. Parse ngày:
from = 2025-03-01T00:00:00
to = 2025-03-31T23:59:59.999999999
4. productId = null → gọi feedbackRepository.findByDateRange(from, to)
SQL: SELECT * FROM feedbacks WHERE created_at BETWEEN '2025-03-01...' AND '2025-03-31...'
Spring tự JOIN attribute_ratings (vì EAGER)
Kết quả: [Feedback{id=1, productId=5, customerId=12, attributeRatings=[AR{attrId=3}, AR{attrId=4}]]
5. Gọi enrich([fb1])
6. Xử lý fb1:
customerCache = {} (rỗng)
computeIfAbsent(12, ...) → key 12 chưa có → chạy lambda:
 UserServiceClient.getCustomer(12)
 → HTTP GET http://localhost:8082/api/customers/12
 → user-service trả về {"fullName": "Nguyễn Văn A"}
 → trả về "Nguyễn Văn A"
customerCache = {12 → "Nguyễn Văn A"}
fb1.setCustomerName("Nguyễn Văn A")
7. Xử lý AR{attrId=3}:
attributeCache = {} (rỗng)
computeIfAbsent(3, ...) → key 3 chưa có → chạy lambda:
 ProductServiceClient.getAttribute(3)
 → HTTP GET http://localhost:8081/api/products/attributes/3
 → product-service trả về {"id":3, "name":"Màu sắc"}
 → trả về "Màu sắc"
attributeCache = {3 → "Màu sắc"}
ar.setAttributeName("Màu sắc")
8. Xử lý AR{attrId=4}:
computeIfAbsent(4, ...) → key 4 chưa có → chạy lambda:
 ProductServiceClient.getAttribute(4)
 → HTTP GET http://localhost:8081/api/products/attributes/4
 → trả về {"id":4, "name":"RAM"}
attributeCache = {3 → "Màu sắc", 4 → "RAM"}
ar.setAttributeName("RAM")
9. Trả về [fb1 đã đầy đủ thông tin]
10. Controller nhận kết quả → Spring serialize sang JSON → gửi về Frontend

JSON response cuối cùng:

```
[
  {
    "id": 1,
    "productId": 5,
    "customerId": 12,
    "customerName": "Nguyễn Văn A",
    "comment": "Tốt lắm!",
    "overallRating": 4,
    "createdAt": "2025-03-15T10:30:00",
    "attributeRatings": [
      {
        "id": 1,
        "attributeId": 3,
        "attributeName": "Màu sắc",
        "rating": 5,
        "comment": "Màu đẹp"
      },
      {
        "id": 2,
        "attributeId": 4,
        "attributeName": "RAM",
        "rating": 4,
        "comment": "RAM ổn"
      }
    ]
  }
]
```

BẢNG TÓM TẮT: HÀM NÀO GỌI HÀM NÀO

Ai gọi	Gọi hàm nào	Mục đích
Frontend	GET /api/feedbacks/range?from=...&to=...	Lấy thống kê
FeedbackController.getByDateRange()	feedbackService.getFeedbacksByDateRange()	Chuyển cho Service
FeedbackService.getFeedbacksByDateRange()	LocalDate.parse(fromDate).atStartOfDay()	Chuyển chuỗi → LocalDateTime
FeedbackService.getFeedbacksByDateRange()	feedbackRepository.findByDateRange()	Truy vấn DB (không có productId)
FeedbackService.getFeedbacksByDateRange()	feedbackRepository.findByProductIdAndDateRange()	Truy vấn DB (có productId)
FeedbackService.getFeedbacksByDateRange()	enrich(list)	Bổ sung tên
FeedbackService.enrich()	customerCache.computeIfAbsent()	Kiểm tra cache trước khi gọi HTTP
FeedbackService.enrich()	userServiceClient.getCustomer(customerId)	Lấy tên khách hàng
FeedbackService.enrich()	attributeCache.computeIfAbsent()	Kiểm tra cache trước khi gọi HTTP

Ai gọi	Gọi hàm nào	Mục đích
FeedbackService.enrich()	productServiceClient.getAttribute(attributeId)	Lấy tên thuộc tính
UserServiceClient.getCustomer()	restTemplate.exchange(url, GET, ...)	Gọi HTTP sang user-service
ProductServiceClient.getAttribute()	restTemplate.exchange(url, GET, ...)	Gọi HTTP sang product-service
user-service nhận request	Tra cứu user_db	Trả về CustomerDto
product-service nhận request	ProductController.getAttributeById()	Trả về AttributeDto
ProductController.getAttributeById()	productService.getAttributeById()	Lấy Attribute từ DB
ProductService.getAttributeById()	attributeRepository.findById()	Truy vấn bảng attributes